

Introduction to Network Traffic Analysis Module

Introduction

Network Traffic Analysis (NTA) can be described as the act of examining network traffic to characterise common ports and protocols used as well as establish a baseline for our environment so that we can accurately and promptly respond to threats. This process helps us detect anomalies and security threats early. Network traffic analysis can also facilitate the process of meeting security guidelines. Some day-to-day use cases of NTA are:

- Collecting real-time traffic to analyse upcoming threats.
- Setting a baseline for normal traffic.
- Identifying and analysing traffic from non-standard ports, suspicious hosts or issues with network misconfigurations.
- Detecting malware such as ransomware.
- Investigation of previous incidents and threat hunting.

If an attacker wishes to breach our network they must inevitably interact with it. Network communication takes place over many different ports and protocols all being utilised concurrently by employees, equipment and customers so to spot malicious traffic we need to rely heavily on statistical methods and our intuition as to what is and is not normal. For example: we may detect many SYN packets on ports that we rarely use in our network so we could conclude that this is someone trying to determine what ports are open on our hosts.

Pre-requisite Knowledge

We do not need to know everything by heart but we should know what to look for when certain aspects of the content seem unfamiliar. This does not just apply to NTA but to all of cybersecurity.

- TCP/IP Stack & OSI Model
- Basic Network Concepts
- Common Ports and Protocols
- Concepts of IP Packets and the Sub-layers
- Protocol Transport Encapsulation

We should have some knowledge of the tools and equipment that we can use to perform NTA, this module only explores a few of these but we should be aware of all them. We should also be aware that most tools can be used for both offensive and defensive purposes.

Tool	Description
tcpdump	A command-line utility that captures and interprets network traffic from an interface or capture file.
Tshark	The command-line version of Wireshark.
Wireshark	A graphical network traffic analyser. It captures and decodes frames off the wire and allows for an in-depth look into the environments. It can run many additional program to provide additional insight about the network packets.
NGrep	A pattern-matching tool similar to grep for Linux. NGrep is designed to work with network traffic patterns. This tool is very useful when trying to debug traffic from HTTP and FTP.
tcpick	A command-line packet sniffer specialising in tracking and reassembling TCP streams.
Network Taps	Taps are devices capable of taking copies of network traffic and sending them to another place for analysis. They can operate actively or passively.
Networking span ports	Span ports are a way to copy frames from layer 2 or 3 networking devices and send them to a collection point.
Elastic Stack	A culmination o tools that can take data from many sources, analyse and visualise it.

BPF Syntax

Many of the above tools have their own syntax and commands to use but one that is shared between them is Berkeley Packet Filter (BPF) syntax. This is the primary method we will use. BPF is a technology that enables a raw interface to read and write from the Data-Link layer. BPF provides filtering and decoding abilities which we will use extensively.

Performing NTA

Performing analysis can be as simple as watching live traffic go by or as complex as capturing data with a tap, sending it to a SIEM and then analysing the pcap data for signatures and alerts. At a minimum, we need to be connected to the network segment we wish to listen on (particularly true for switched environments), our capture device should be connected to the

same network. Devices like network tap, span ports and port mirroring can help us get copies of all traffic regardless of what segment it is on. NTA is a very dynamic process and is not a direct loop - it is greatly influenced by what we are looking for:



We need to begin capturing traffic before we can do anything else. If we know what we are looking for we can use capture filters. We can then filter out more unnecessary traffic to make analysis easier. We can start looking at specific hosts, ports and TCP header flags. Some key questions to ask as this stage would be:

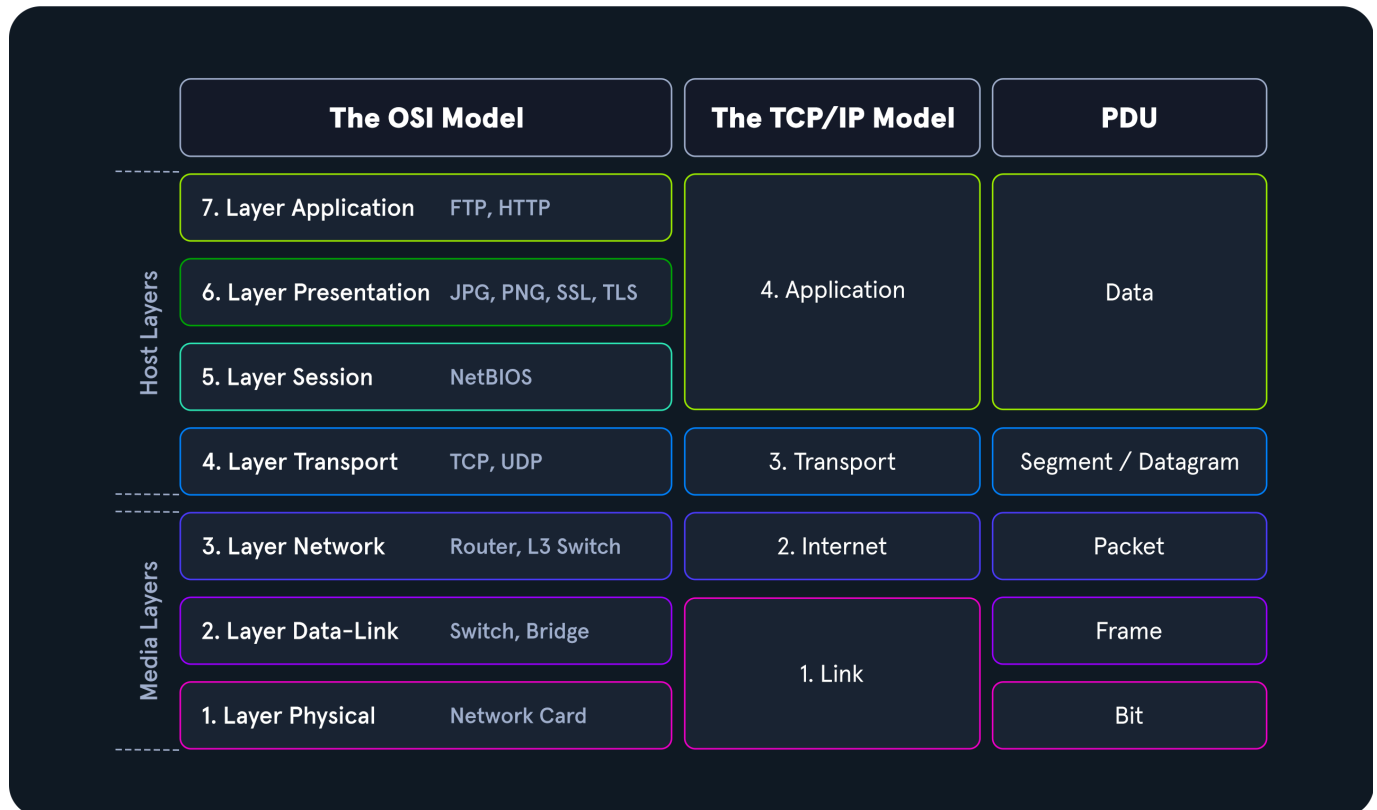
- Is the traffic encrypted and should it be?
- Can we see users attempting to access resources that they should not have access to?
- Are there different hosts talking to each other when they typically do not?
- Are we seeing any errors / Is a device not responding how it should?

If we spot any issues at any point we should follow up and attempt to make a change to fix the problem.

Networking Primer

Throughout this module we will need to have a solid understanding of networking and what behaviour is normal for a network. If we do not have this knowledge we cannot accurately analyse any traffic that we may capture.

OSI / TCP-IP Models



Layers 1-4 of the OSI model are focused on controlling the transportation of data between hosts. Layers 5-7 handle the interpretation, management and presentation of the data presented to the end-user. The OSI model can be thought of as the theory behind networking whereas the TCP/IP stack is more closely aligned with the actual functionality of networking. Throughout this module we will examine many different protocol data units (PDUs), so a functional understanding of how it works is required. A PDU is a data packet made up of control information and data encapsulated from each layer of the OSI model.

When inspecting a PDU we need to keep track of the idea of encapsulation in mind. As our data moves down the protocol stack, each layer will wrap the previous layers' data in a new bubble we call encapsulation. This adds the necessary information of that layer into the header of the PDU, this information can vary by level but it includes what is held by the previous layer, operational flags, any options required, the source and destination IP addresses, ports, transport and application layer protocols.

Consider that when viewing packets in NTA software such as Wireshark, the PDU will show in the reverse order because that is the order that it has been decoded in.

Addressing Mechanisms

With MAC-Addressing each logical or physical interface attached to a host has a 48-bit, six octet address representation in hexadecimal format. Media Access Control (MAC) addressing is utilised in layer 2 communications. If the layer 2 traffic needs to cross a layer 3 interface, it is sent to the layer 3 egress interface which routes it to the correct network. This means inspecting the layer 2 traffic will show as the destination being the router interface.

With IPv4 addressing, we can route packets across networks to hosts located outside our immediate vicinity. An IPv4 address is made up of a 32-bit, four octet number represented in decimal format. Each octet of an IP address is represented by a number ranging from 0 to 255. When examining a PDU the IP addresses will appear in layer 3 of the OSI model and layer 2 of the TC/IP model.

One of the problems with IPv4 is the lack of possible IP addresses, especially given large chunks of address space sectioned off for special use or private addressing. To solve this issue, variable-length subnet masks (VLSM) and classless inter-domain routing (CIDR) was implemented. This allowed us to redefine the usable IP addresses, changing how addresses are assigned to users. The other thing that was done was the development of IPv6. IPv6 provides a much larger address space by being a 128-bit, 16 octet address represented in hexadecimal format. IPv6 also comes with other benefits such as better support for multicasting and enhanced security.

IPv6 uses four main types of addresses:

- Unicast - addresses for a single interface
- Anycast - addresses for multiple interfaces, where only one of them receives the packet.
- Multicast - addresses for multiple interfaces where all of them receive the same packet.
- Broadcast - Does not truly exist and is realised with multicast addresses.

The main issue with IPv6 is the adoption rate.

TCP / UDP, Transport Mechanisms

The transport layer has several mechanisms to help ensure seamless delivery of data. Application data from the higher layers traverse down the stack to the transport layer which directs how the traffic will be encapsulated and thrown to the lower layer protocols. Once the data reaches its recipient, the transport layer is responsible for reassembling the data back in the correct order. The two mechanisms used to accomplish this are the transmission control and user datagram protocol (TCP and UDP respectively).

Characteristic	TCP	UDP
Transmission	Connection-oriented	Connectionless
Connection Establishment	Three-way handshake.	Does not ensure the destination is listening.
Data Delivery	Stream-based conversations	Packet by packet.
Receipt of Data	Sequence and acknowledgement numbers used to account for data.	None.
Speed	Slower than UDP due to more overhead and configuration.	Fast.

TCP is a much more reliable protocol and is used when moving data that requires completeness such as when setting up an SSH connection. UDP is less reliable but is faster and so is used when speed is more important than completeness. For example we may use UDP when streaming a video since we would care more if we were constantly buffering than if a pixel gets dropped. Another example would be sending a DNS request.

TCP Three-way Handshake

One of the ways TCP ensures the delivery of data from server to client is the utilisation of sessions. These sessions are established through what is called a three-way handshake. To make this happen, TCP utilises an option called flags:

1. The client sends a packet with the SYN flag set and any other negotiable options in the TCP header
2. The server will respond with a TCP packet that includes a SYN flag set for the sequence number negotiation and an ACK flag set to acknowledge the previous packet sent by the host.
3. The client will respond with a TCP packet with an ACK flag set agreeing to the negotiation.

At the end of communication, a similar process occurs:

1. The client sends a packet with the FIN and ACK flags set.
2. The server sends back FIN, ACK.
3. The client sends a final packet acknowledging the communication is over. ACK.

HTTP

Hypertext Transfer Protocol is a stateless Application Layer protocol that has been in use since 1990. HTTP enabled the transfer of data in clear text between a client and a server over TCP. The client would send an HTTP request to the server, a session is established and the server responds with the requested media. HTTP uses ports 80 or 8000 over TC during normal operations. To perform operations with HTTP relies on methods which define the actions taken when requesting a URI, these methods are:

Method	Required*	Description
HEAD	yes	A method that requests a response similar o a get request except the message body is not included. Used to get more information about a server.
GET	yes	The most common method. Requests information and content from the server.
POST	no	A way to submit information to a server based on the fields in the request. The actual action taken will vary based on the server so we should pay attention to the response codes sent to validate the action.
PUT	no	Takes the appended message data and places it under the requested URI. If an item does not already exist is will create one. Often used for uploading files.
DELETE	no	Removes the object at the given URI.
TRACE	no	Allows for remote server diagnostics.
OPTIONS	no	Used to gather information on the supported HTTP methods. This can help us identify the requirements for interacting with a specific resource or server without actually requesting data or objects from it.
CONNECT	no	Reserved for use with proxies or security devices. Allows tunnelling over HTTP.

*Methods that are required must work with all HTTP implementations.

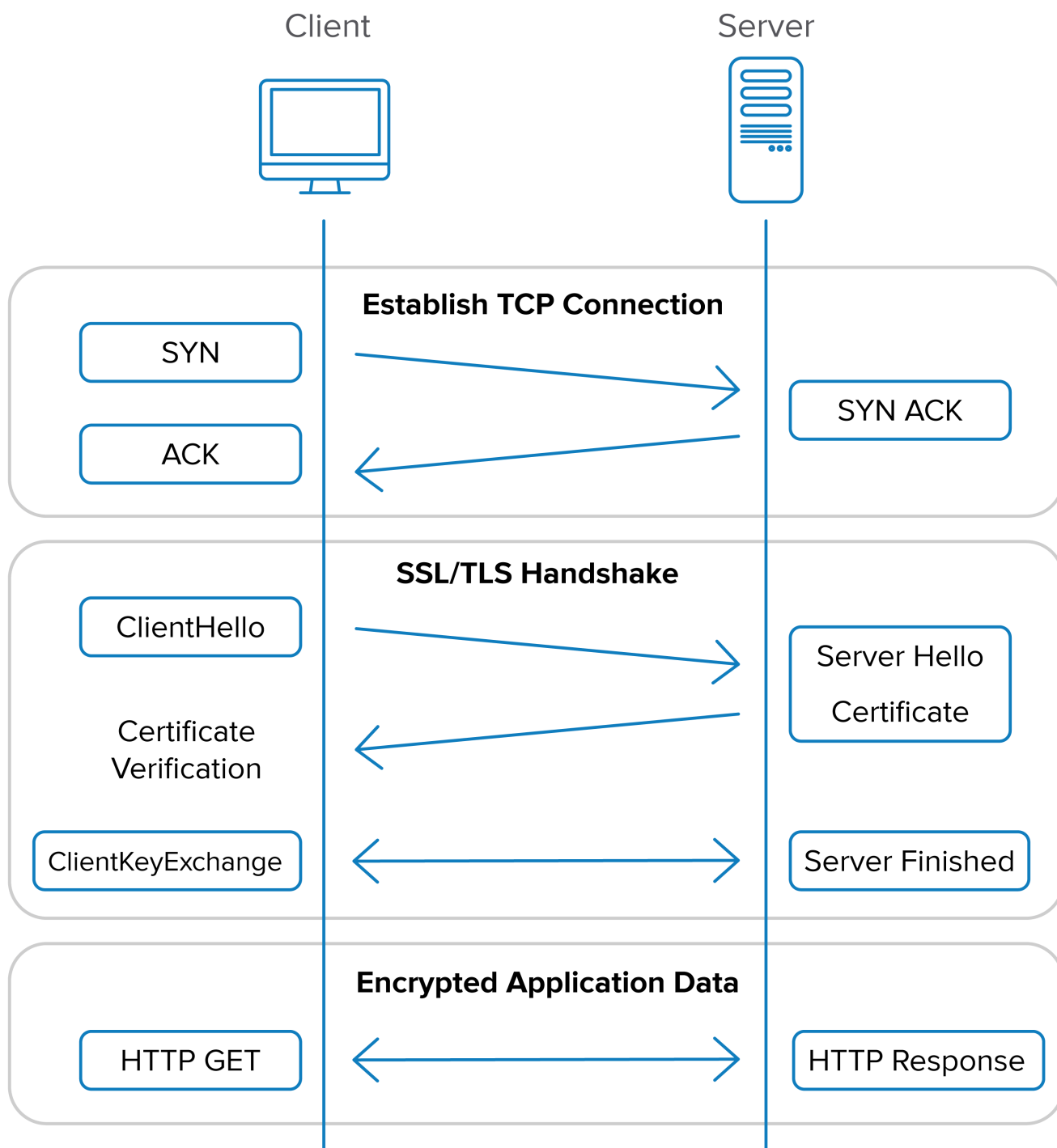
HTTPS

HTTP Secure is a modification of the HTTP protocol designed to use transport layer security (TLS) or secure sockets layer (SSL). TLS is used as an encryption mechanism to secure the communications between a client and a server. TLS can wrap regular HTTP traffic in TLS meaning we can encrypt our entire conversation and not just the data sent or requested.

Although HTTPS is fundamentally HTTP at the base, HTTPS uses ports 443 and 8443 instead of the standard HTTP ports. In the first few packets we can see that the client establishes a session to the server using port 443, once the session is initiated a TLS ClientHello is sent next to begin the TLS handshake. During the handshake several parameters are agreed upon. Once the session is established, all data and methods will be sent through the TLS connection and appear as TLS application data. TLS is still using TCP as its transport protocol so we will still see acknowledgement packets coming over port 443. The handshake works as follows:

1. The client and server exchange hello messages to agree on connection parameters.
2. Client and server exchange necessary cryptographic parameters to establish the secret.
3. Client and server exchange certificates and information allowing them to authenticate within the session.
4. Generate a master secret using the premaster secret and random values.
5. Client and server issue negotiated security parameters to the record layer portion of the TLS protocol.
6. Client and server verify that their peer has calculated the same security parameters and that the handshake occurred without tampering.

Encryption is in itself a complex and lengthy topic that goes way beyond this simple explanation. It might be worthwhile revising this.



FTP

File transfer protocol is an application layer protocol that enables quick data transfer between computing devices. FTP can be used from the command line, web browser or through a graphical interface. FTP is established as an insecure protocol and most users have moved tools to secure channels. Most web browsers have a phased out support for FTP. When we think about communication between hosts, we typically think about a client and server talking over a single socket. Through this socket, both the client and server send commands and data

over the same link. In this aspect, FTP is unique since it uses multiple ports at a time, FTP uses ports 20 and 21 over TCP. Port 20 is used for data transfer and port 21 is used for issuing commands controlling the FTP session. FTP supports both user and anonymous authentication.

FTP is capable of running in two different modes, active and passive. Active is the default mode and means the server listens for a control command port from the client, stating what port to use for transfer. Passive mode enables us to access FTP servers location behind firewalls or a NAT-enabled link that makes direct TCP connections impossible. In this instance the client sends the PASV command and waits for a response from the server informing the client what IP and port to use for the data transfer channel connection.

The most common commands what will get sent over port 21 are:

- USER: Specifies the user to log in as.
- PASS: sends the password for the user trying to log in.
- PORT: when in active mode, changes the data port.
- PASV: switches the connection to passive mode.
- LIST: lists all files in the current directory.
- CWD: changes the current directory to the one specified.
- PWD: displays the current directory.
- SIZE: returns the size of the specified file.
- RETR: retrieves the file from the FTP server.
- QUIT: exits the session.

SMB

Server Message Block (SMB) is a protocol widely seen in windows environments that enables sharing resources between hosts over common networking architectures. SMB is a connection-oriented protocol that requires user authentication from the host to the resource to ensure that the user has the correct permissions to use that resource. In the past, SMB used NetBIOS as its transport over port 445, NetBIOS over TCP port 139 and the QUIC protocol. As a user, SMB provides us easy and convenient access to resources like printers, shared drives, authentication servers and more. Like any application that uses TCP as its transport mechanism, it will perform standard functions like the three-way handshake.

The Analysis Process

NTA is a dynamic process that can change depending on the tools we have on hand, the permissions given to us by the organisation and our network's visibility. Our goal is to provide a repeatable process. Traffic analysis is a detailed examination of an event or process, determining its origin and impact, which can be used to trigger specific precautions.

We need to break down the data into understandable chunks, examining it for anything that deviates from regular network traffic to detect any potentially malicious communications. Traffic analysis is a highly versatile tool in our defensive toolbox. In more advanced implementations for NTA that include other tool like IDS/IPS, firewalls and SIEMs having additional traffic data is invaluable. Watching live network traffic can help us diagnose connection issues and see if all protocols are operating correctly. This is a dynamic skill and using automated tools to aid us is perfectly fine, but we cannot rely on them solely.

Analysis Dependencies

Traffic capturing and analysis can be performed in two different ways, active or passive. Each has its dependencies. For active (or in-line) traffic capture it is up to us if we perform the capture and analysis separately or in real-time.

Dependencies	Operation Type	Description
Permission	Passive/Active	Depending on the organisation we are working in, capturing data may be against policy or even against the law. We need to ensure we have permission.
Mirrored Port	Passive	A switch or router interface configured to copy data from other sources to that specific interface along with the capability to place your NIC into promiscuous mode.
Capture Tool	Passive/Active	A way to ingest the traffic. A computer with access to software tools can be sufficient. Consider that NTA is a resource intensive procedure.
In-line placement	Active	Placing a tap in-line requires a topology change for the network you are working in. For the sake of routing and switching it will be an invisible hop the traffic passes through on its way to the destination.
Network Tap or Host with multiple NIC's	Active	A computer with two NIC's or a device such as network tap is required to allow the data to flow. The best place for a tap is in a layer 3 link that allows for the capture of any traffic routing outside the local network.
Storage and Processing Power	Passive/Active	As mentioned already, NTA is very resource intensive. Make sure you filter your data otherwise it will be impossible to manage.

It is always important to have a network baseline so that we can filter off known good communications. Using data analysis tools in Wireshark can help us identify hosts that might be sending a large amount of data. We can also filter for uncommon ports.

Descriptive Analysis

Descriptive analysis is an essential step in any data analysis. It services to describe a data set based on individual characteristics. It also helps deect potential errors in collection and outliers in the data set. We aim to answer some questions:

- What is the issue?
- What are we looking for?
- What time period has/will this issue occur at?
- What are our targets, hosts and protocols?

Descriptive analysis will help us cover some of these critical concepts.

Diagnostic Analysis

Diagnostic analysis clarifies the causes, effects and interactions of conditions. In doing so, it provides insights that are obtained through correlation. We are aiming to find reasons.

- We need to capture network traffic.
- We need to identify required traffic components via filtering.
- We need to hunt for targets.

By capturing traffic around the source of the issue and clearing out known good data we can determine if it is the cause of the problem. We want to validate the cause of our problems.

Predictive Analysis

Evaluating historical and current data, predictive analysis creates a model for future probabilities. This method of analysis makes it possible to identify trends and detect deviations from expected behaviour.

- Take notes and map found results
- Present a summary of our analysis, what did we find?

By performing an evaluation of the data and comparing it to our baseline we can search for markers of infiltration or exploitation.

Components of Effective Analysis

1. Know your environment: There are several key components to perform NTA successfully. We need to keep asset inventories and network maps.
2. Placement is key: Our host placement for capturing traffic is a critical thing. We want to be as close as possible to the source of the issue.
3. Persistence: The most crucial component for us is persistence. The issue may not always be easy to spot. It may not be a frequent event on the network. Do not lose the drive to find the problem.

Analysis Approach

We can start by looking for some easy wins. We can look at the standard protocols and work our way into the more specific ones for our organisation. Most attacks will come from the internet, so it has to access the internal net somehow - this means there will be logs. After this, check standard communication protocols and be mindful of the organisations security plan. Look for patterns. Is a specific host or set of hosts checking in with something on the internet at the same time daily? Check anything host to host within the network. Look for unique events. Ask other people to look over the same data.

Tcpdump Fundamentals

Tcpdump is a command-line packet sniffer that can directly capture and interpret data frames from a file or network interface. It is designed to run on Unix systems and previously had a windows port called WinDump. It is a straightforward tool that can be used through any terminal or remote connection. To capture network traffic "off the wire", it uses the pcap and libpcap libraries and a network interface in promiscuous mode to listen for data intended for any network device (not just ours). Due to the direct access required to the hardware, we need to run this tool with root or administrator privileges. To view the location of the package on our Linux machine we can run the following commands:

```
which tcpdump
```

```
sudo apt install tcpdump
```

Because of the many different functions and filters, we need to familiarise ourselves with the tool's essential features. Some of the most common flags are given below:

Switch	Result
D	Display an interfaces available to capture from.
i	Select an interface to use.
n	Do not convert addresses to names.
e	Grab the Ethernet header.
X	Show contents of packets in hex and ASCII
XX	Equivalent to -X -e.
v, vv, vvv	Increase verbosity of output.
c	Grab a specific number of packets.
s	Defines how much of a packet to grab.
q	Print less protocol information.
r file.pcap	Read from a file
w file.pcap	Write to a file

To view the complete list of switches we can view the man page:

```
man tcpdump
```

Some examples of running the command are given below:

```
sudo tcpdump -i eth0
```

```
sudo tcpdump -i eth0 -nn
```

```
sudo tcpdump -i eth0 -nnvXX
```

When looking at the output of tcpdump, it can be quite overwhelming, especially given the variety of views give by applying different flags.

```

17:39:26.500472 IP 172.16.146.2.21 > 172.16.146.1.49769: Flags [P.], seq 1:77, ack 17, win 509, options
[nop,nop,TS val 428627084 ecr 3427972529], length 76: FTP: 220 Welcome to the PowerBroker FTP service. G
rab or leave juicy info here.
  0x0000: 4502 0080 4405 4000 4006 7a4c ac10 9202 E...D.@.@.zL...
  0x0010: ac10 9201 0015 c269 a462 9f3b 547a afe1 .....i.b.;Tz..
  0x0020: 8018 01fd 7c97 0000 0101 080a 198c 548c ....|.....T.
  0x0030: cc52 b5b1 3232 3020 5765 6c63 6f6d 6520 .R..220.Welcome.
  0x0040: 746f 2074 6865 2050 6f77 6572 4272 6f6b to.the.PowerBrok
  0x0050: 6572 2046 5450 2073 6572 7669 6365 2e20 er.FTP.service..
  0x0060: 4772 6162 206f 7220 6c65 6176 6520 6a75 Grab.or.leave.ju
  0x0070: 6963 7920 696e 666f 2068 6572 652e 0d0a icy.info.here...
17:39:26.500711 IP 172.16.146.2.21 > 172.16.146.1.49769: Flags [P.], seq 77:111, ack 17, win 509, option
s [nop,nop,TS val 428627084 ecr 3427972529], length 34: FTP: 331 Please specify the password.
  0x0000: 4502 0056 4406 4000 4006 7a75 ac10 9202 E..VD.@.@.zu...
  0x0010: ac10 9201 0015 c269 a462 9f87 547a afe1 .....i.b..Tz..
  0x0020: 8018 01fd 7c6d 0000 0101 080a 198c 548c ....|m.....T.
  0x0030: cc52 b5b1 3333 3120 506c 6561 7365 2073 .R..331.Please.s
  0x0040: 7065 6369 6679 2074 6865 2070 6173 7377 pecify.the.passw
  0x0050: 6f72 642e 0d0a ord...
17:39:26.500830 IP 172.16.146.1.49769 > 172.16.146.2.21: Flags [.], ack 77, win 2057, options [nop,nop,T
S val 3427972532 ecr 428627084], length 0
  0x0000: 4500 0034 0000 0000 4006 fe9f ac10 9201 E..4....@.....
  0x0010: ac10 9202 c269 0015 547a afe1 a462 9f87 .....i..Tz...b..
  0x0020: 8010 0809 f7aa 0000 0101 080a cc52 b5b4 .....R..
  0x0030: 198c 548c ..T.

```

The first piece of information is highlighted with yellow in the image above and is the timestamp. This field is configurable to show the date and time in a suitable format for our SIEM.

In orange is the protocol, source and destination address of the packet along with the port number used to connect.

Marked in green are any flags used.

Red shows the sequence and acknowledgement numbers used to track the TCP segment.

Blue shows any negotiated TCP values established between the client and the server.

Finally in white is the any other notes the dissector found. This could include more header information.

We can filter and sort through the output of tcpdump using builtin bash commands or set up scripts to hunt for suspicious behaviour automatically. We can set up tcpdump to write to a capture file, just consider that these files can fill up quickly and take up a large amount of storage space.

Filtering and Advanced Syntax Options

Using more advanced filtering options like the ones listed below will enable us to trim down what traffic is printed output or sent to file. By reducing the amount of information we capture we can reduce the space needed to write the file and help process the data quicker. We can also aim to mitigate alert fatigue. The filters below are by no means an exhaustive list they were chosen because they are the most frequently used and will get us up and running quickly.

Filter	Result
host	Filters based on the designated host.
src / dest	Designate a source/destination host or port.
net	Will show us any traffic sourcing from or destined to the designated network. Uses / notation.
proto	Filter by a specific protocol type.
port	Shows all traffic with the specified port as either source or destination.
portrange	Allows us to specify a range of ports.
less / greater	Can be used to filter on size.
and / &&	Used to concatenate.
or	Allows for a match on either one of multiple conditions.
not	Matches the negation of the filter.

```
sudo tcpdump -i eth0 host 172.16.146.2
```

```
sudo tcpdump -i eth0 src host 172.16.146.2
```

```
sudo tcpdump -i eth0 tcp src port 80
```

```
sudo tcpdump -i eth0 dest net 172.16.146.0/24
```

```
sudo tcpdump -i eth0 proto 17
```

```
sudo tcpdump -i eth0 portrange 0-1024
```

```
sudo tcpdump -i eth0 less 64
```

```
sudo tcpdump -i eth0 host 192.168.0.1 and port 23
```

```
sudo tcpdump -r sus.pcap icmp || host 172.16.146.1
```

```
sudo tcpdump -r sus.pcap not icmp
```

Pre-capture vs Post-capture Processing

There are pros and cons for applying filters either before or after capturing data. Applying filters first lets us hugely reduce the footprint of the capture files that we are saving. Applying filters after capturing allows us to run many different filters and views on the file without actually editing or changing the file at all.

We can dig as deep as we wish into the packets we have captured, it just requires knowledge of how the protocols are structured. For example, if we only want to see packets with the TCP SYN flag set, we can use the following cards:

```
sudo tcpdump -i eth0 'tcp[13] &2 != 0'
```

```
sudo tcpdump -i eth0 'tcp[tcpflags] & (tcp-syn|tcp-ack) != 0'
```

Analysis Questions

Whenever we perform traffic analysis there are some questions we should ask ourselves to keep on track:

- What type of traffic is there?
- Is there more than one conversation?
- How many unique hosts are there?
- What is the first conversation?
- What traffic can i filter out?
- What are the servers in the conversations?
- What records were requested or methods used?

Wireshark

Wireshark is a free and open source network traffic analyse like tcpdump, with a graphical interface. Wireshark is multi-platform and capable of capturing live data off many different

interface types and saving the traffic to several formats. Wireshark comes with decryption capabilities for common standards.

```
which wireshark
sudo apt install wireshark
```

Wireshark also comes in CLI form called Tshark. Tshark is useful on machines with little to no desktop environment and can also be used to easily pipe output to another program. To view all of the available Tshark switches you can load it's help page:

```
tshark -h
```

Some common switches are given below:

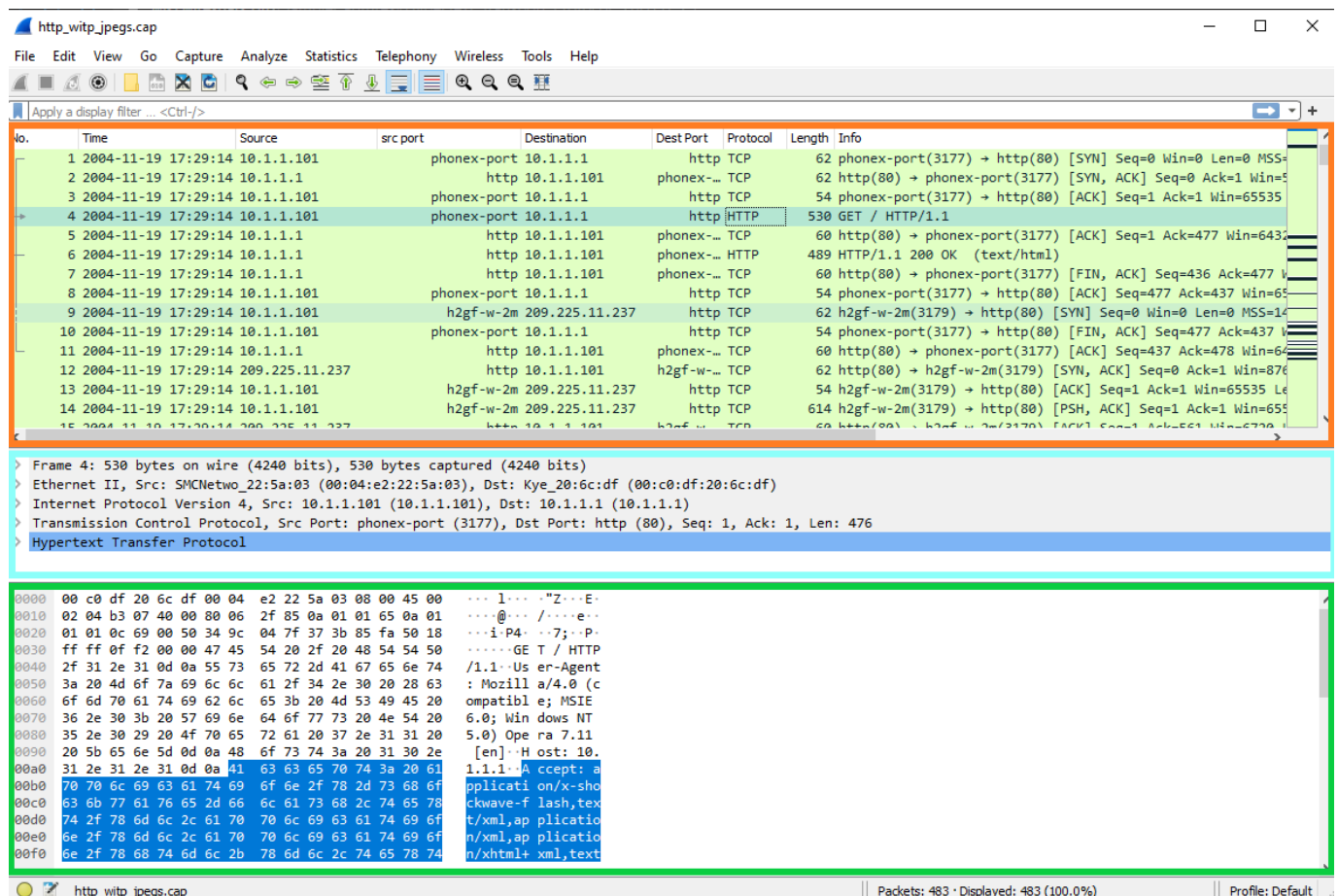
Switch	Result
D	Display all interfaces and then exit.
L	List the link-layer mediums (e.g ethernet) you can capture from then exit
i	Select capture interface.
f	Packet filter in libpcap syntax.
c	Grab a specific number of packets and then exit.
a	Defines an autostop condition.
r	Read from a file
w	Write to a file
P	Print packet summary while writing to file
x	Add hex and ASCII output to the capture

Notice that a lot of these switches are the same as with tcpdump.

```
tshark -i 1 -w test.pcap
```

```
sudo tshark -i eth0 -f "host 172.16.146.2"
```

Termshark is a TUI application that provides a Wireshark-like interface in the terminal window. There are 3 main panes to the Wireshark window:



Surrounded in orange is the packet list. This contains a summary line of each packet along with customisable columns. The Packet details are in blue and provide more information about each packet, in particular it shows us the chunks we would expect from the OSI model. In green we can see the packet bytes window, this allows us to see the packet contents in ASCII and hex output.

While capturing traffic in Wireshark, we have several options regarding how and when we filter out traffic - we can use either capture or display filters. Capture filters are entered before the capture is started and they use BPF syntax. This is a good way to trim down the conversation before saving it to disk. We can write our own filters or we can use the builtin ones, which we can see from the capture filters tab in Wireshark.

Display filters are used while the capture is running and after the capture has stopped. These are proprietary to Wireshark and there are many different options for almost any protocol.

Filter	Result
<code>ip.addr == x.x.x.x</code>	Show traffic pertaining to a certain host.
<code>ip.addr == x.x.x.x/24</code>	Show traffic pertaining to a specific network.
<code>dns / tcp / ftp / arp / ip</code>	Filter by protocol.
<code>ip.src/dst == x.x.x.x</code>	Traffic to/from a specific host.
<code>tcp.port == x</code>	Filter by tcp port.
<code>tcp.port / udp.port != x</code>	Filter by everything but the port specified.
<code>and / or / not</code>	Standard logical operators.

Advanced Usage

Wireshark comes with many additional capabilities including tracking TCP conversations and cracking WiFi credentials. The statistics and analyse tabs can provide us with additional insights into the data we are examining. The plugins here can give us detailed reports about the network traffic being used. From the analyse tabs we can use plugins that allow us to do things such as follow TCP streams, filter on conversation types, prepare new packet filters and examine the expert info Wireshark generates about this traffic.

Wireshark can stitch TCP packets back together to recreate the entire stream in a readable format. This also allows us to pull data out of the capture. To use this feature, right click on a packet from the stream we wish to recreate, select follow TCP and read the conversation in the new tab that opens up. Alternatively we can use the filter `tcp.stream eq #` to find and track conversations.

Wireshark can recover many different types of data from streams. This feature requires you to have captured the entire conversation, otherwise Wireshark will fail to put the data back together. To extract files from a stream you need to stop your capture, select file then export then the protocol to extract from. Another way to grab data out of the pcap file comes from FTP. Wireshark has several filters for FTP connections that we can use to take a closer look at the traffic.

```
ftp
```

```
ftp.request.command
```

```
ftp-data
```

Since FTP uses TCP as its transport mechanism, we can once again use the follow tcp stream function of Wireshark. The basic steps to pull FTP data from a pcap are:

- Identify any traffic using the ftp filter
- Look for command controls to see if anything was transferred and on what devices.
- Choose a file, then filter for ftp-data. Select a packet that corresponds with our file of interest and follow the TCP stream that correlates to it.
- Change "Show and save data as" to "Raw" and save the content as the original filename.

Decrypting RDP connections

If we have the required key used between two hosts for encrypting traffic, Wireshark can decrypt it for us. By default RDP uses TLS to encrypt the data which means it can be harder to spot. One thing we can do is instead of filtering for the RDP protocol, we can filter for the 3389 RDP port. To add an RDP-key to Wireshark we go into preferences, Protocols, TLS and edit RSA keys list.

Useful Resources

- [Protocol numbers](#)
- [RFC links](#)
- [Termshark](#)